

Status codes & error handling

COMPLETE LEARNING GUIDE

What You'll Learn

HTTP status codes are a contract: they let a client know, without parsing the response body, whether a request succeeded, failed because of something the client did wrong, or failed because of something on the server — and clients, proxies, caches, and monitoring tools all rely on that contract being honored accurately. A shockingly large number of production APIs return 200 OK for everything, including errors, forcing every client to parse the body just to find out whether the call actually worked. This guide covers the status code families in depth, how to design a consistent JSON error response shape, and how to centralize error handling so that every endpoint in an application produces errors in exactly the same format, rather than each route inventing its own.

1. 2xx: Success, But Which Kind?

Not all successes are the same, and using the right 2xx code communicates useful information a generic 200 doesn't. 200 OK is the general-purpose success response, typically used for GET, PUT, and PATCH. 201 Created should be returned specifically from a successful POST that creates a new resource, ideally alongside a Location header pointing to the new resource's URL. 202 Accepted signals that a request has been accepted for asynchronous processing but isn't complete yet (common in queued/background-job APIs). 204 No Content is used when a request succeeds but there's nothing meaningful to return — a common choice for a successful DELETE.

- 200 OK — general success, response has a body.
- 201 Created — a POST successfully created a resource; include a Location header.
- 202 Accepted — request accepted for async/background processing, not yet complete.
- 204 No Content — success, but no response body (typical for DELETE).

2. 4xx: The Client Did Something Wrong

The 4xx family tells the client the request itself needs to change before retrying it as-is will help. 400 Bad Request is the generic 'malformed request' code. 401 Unauthorized actually means 'not authenticated' — the request has no valid credentials at all. 403 Forbidden means 'authenticated, but not allowed' — the server knows who you are and is refusing anyway. 404 Not Found means no resource exists at that URL. 405 Method Not Allowed means the resource exists but doesn't support the HTTP method used. 409 Conflict signals the request conflicts with the current state of the resource (e.g. a duplicate unique field). 422 Unprocessable Entity signals the request was well-formed but failed semantic/business validation. 429 Too Many

Requests signals rate limiting.

- 400 Bad Request — malformed syntax, missing required field, invalid JSON.
- 401 Unauthorized — no valid credentials provided at all (really means 'unauthenticated').
- 403 Forbidden — credentials are valid, but the user lacks permission for this action.
- 404 Not Found — no resource at this URL (or, deliberately, hiding the existence of one the user can't access).
- 409 Conflict — request conflicts with current resource state (e.g. duplicate email on signup).
- 422 Unprocessable Entity — syntactically valid but fails validation rules (invalid email format, out-of-range value).
- 429 Too Many Requests — the client has been rate-limited.

3. 5xx: The Server Did Something Wrong

5xx codes tell the client the failure is on the server's side, and that retrying the exact same request might succeed later without any change on the client's part. 500 Internal Server Error is the generic catch-all for an unhandled exception. 502 Bad Gateway means a server acting as a proxy/gateway got an invalid response from an upstream server. 503 Service Unavailable means the server is temporarily unable to handle the request (overloaded, or down for maintenance) — often paired with a Retry-After header. Client code should generally be written to retry 5xx errors with backoff, but never retry 4xx errors without first fixing the request that caused them.

4. A Consistent Error Response Shape

Beyond choosing the right status code, every error response in an API should share one consistent JSON shape, so client code can handle errors generically instead of writing bespoke parsing per endpoint. A good shape includes a stable machine-readable error code (for programmatic handling), a human-readable message (for logs/debugging), and optionally a list of field-level validation details for 422 responses. The status code goes in the HTTP response line itself — it should not be duplicated as a string inside the body.

```
JSON
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "The 'email' field must be a valid email address.",
    "details": [
      { "field": "email", "issue": "invalid_format" }
    ]
  }
}
```

5. Centralizing Error Handling in Express

Rather than formatting error responses inline in every route, define a small custom error class carrying a `statusCode`, and funnel every error (thrown or passed to `next()`) through one centralized error-handling middleware registered at the very end of the app. This guarantees every endpoint's errors look identical, and it's the only place that needs to change if the error shape is ever revised. In production, this handler should also make sure raw error messages and stack traces from unexpected 500s never reach the client.

JAVASCRIPT

```
class ApiError extends Error {
  constructor(statusCode, code, message, details) {
    super(message);
    this.statusCode = statusCode;
    this.code = code;
    this.details = details;
  }
}

app.post("/api/users", (req, res, next) => {
  if (!req.body.email) {
    return next(new ApiError(422, "VALIDATION_ERROR", "The 'email' field is required.));
  }
  // ...
});

// centralized error handler, registered last
app.use((err, req, res, next) => {
  const status = err.statusCode || 500;
  console.error(err);
  res.status(status).json({
    error: {
      code: err.code || "INTERNAL_ERROR",
      message: status === 500 ? "Internal Server Error" : err.message,
      details: err.details,
    },
  });
});
```

6. 400 vs. 422: A Practical Line to Draw

The distinction between 400 and 422 causes endless debate, but a practical rule resolves most cases: use 400 when the request is malformed at the syntax/structure level (invalid JSON, wrong data type, a required field missing entirely), and use 422 when the request is well-formed but fails a semantic or business rule (an email field that's present and a string but not a valid email address, a date that's in the past when it must be in the future). Picking one consistent rule and documenting it matters more than which exact rule is chosen — the goal is that client developers can predict which code they'll get.

7. Never Leak Internal Details in Production Error Responses

A stack trace, database connection string fragment, or internal file path accidentally included in a 500 response body is a real security exposure, handing an attacker information about the server's internals. Development environments can afford verbose error responses for debugging, but production environments must return a generic message for unexpected (500-class) errors, while still logging the full detail server-side (to console, a log aggregator, or an error-tracking service like Sentry) for developers to investigate.

Common Pitfalls

- Returning 200 OK for every response, including errors, and encoding success/failure only inside the JSON body — forcing clients to parse the body just to know if something failed.
- Using 404 for authentication and authorization failures instead of 401/403, or inconsistently mixing the two approaches across an API.
- Returning 500 for validation errors that are really the client's fault (should be 400/422), making retries look pointless when they'd actually succeed with corrected input.
- Inventing a different error response JSON shape per endpoint or per developer, so client code can't handle errors generically.
- Confusing 401 (not authenticated at all) with 403 (authenticated but not permitted) — returning the wrong one misleads clients about whether logging in again would help.
- Leaking stack traces, internal error messages, or database details in production error responses.
- Not setting Retry-After on 429/503 responses, leaving well-behaved clients to guess how long to back off.

Key Takeaways

- Status codes are a contract — 2xx/4xx/5xx tell the client success, client-fault, or server-fault before it even parses the body.
- 401 means 'who are you?' (not authenticated); 403 means 'I know who you are, and no' (not authorized) — never conflate them.
- Use a consistent JSON error shape (code, message, optional details) across every endpoint in an API.
- Centralize error handling in one place (e.g. Express's error-handling middleware) rather than formatting errors inline in every route.
- 5xx errors are safe to retry; 4xx errors should be fixed on the client side before retrying.
- Never return internal error details (stack traces, DB info) to the client in production — log them server-side instead.

Learn More

MDN — HTTP Response Status Codes

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>

RESTful API — HTTP Status Codes

<https://restfulapi.net/http-status-codes/>

Express.js — Error Handling

<https://expressjs.com/en/guide/error-handling.html>